

Generation-for-nlp Wrap-up Report

1. 프로젝트 개요

1.1. 배경 및 연구 목적

본 프로젝트는 "제한된 컴퓨팅 자원 하에서 Small LLM의 수능형 문제 풀이 성능을 어디까지 극대화할 수 있는가?"라는 핵심 질문에서 출발하였다. 특히 한국어라는 언어적 특수성과 수능이라는 고유한 문제 유형을 고려하여, 다양한 요소에서의 성능 향상을 추구했다.

프로젝트의 최종 목표는 한국어 도메인 지식이 요구되는 수능형 객관식 문제를 효과적으로 해결할 수 있는 최적의 파이프라인을 구축하는 것이다. 이를 위해 데이터 증강, 최적의 모델 선정, 손실 함수 최적화, 그리고 다중 에이전트 협업 등 다양한 실험적 방법론을 적용하고 검증하였다.

1.2. 프로젝트 수행 방향성 및 컨셉

우리 팀은 프로젝트를 수행함에 있어 다음과 같은 세 가지 핵심 전략을 수립하였다.

- Data-Centric AI:** 모델 아키텍처의 수정보다 선행되어야 할 것은 데이터의 품질과 다양성 확보이다. 기존 데이터셋의 편향성을 분석하고, 부족한 도메인 데이터를 확보하기 위해 멀티모달 모델을 활용한 자동화된 데이터 구축 파이프라인을 설계하였다.
- Efficiency & Reproducibility:** 제공된 V100 GPU 환경의 한계를 극복하기 위해 양자화 기술과 메모리 효율적 학습 라이브러리를 도입하여, 하드웨어 제약 내에서 최대 성능을 이끌어내고자 하였다.
- Hypothesis-Driven Experiment:** 단순히 성능을 높이기 위한 무작위 실험이 아닌, 명확한 문제 정의에 기반한 가설을 설정하고, 이를 검증하는 방식으로 실험을 설계하여 논리적 완결성을 높였다.

1.3. 활용 장비 및 개발 환경

본 연구의 모든 실험은 재현성을 보장하기 위해 다음과 같은 환경에서 수행되었다.

- Hardware:** NVIDIA V100 GPU (32GB VRAM) x 3
- Frameworks:** PyTorch, HuggingFace Transformers
- Core Libraries:**
 - Unsloth:** 학습 메모리 최적화 및 속도 향상
 - Bitsandbytes:** 4-bit 양자화(QLoRA) 지원
 - WandB (Weights & Biases):** 실험 로그 추적 및 시각화
 - Pypdfium2 & GPT-4o API:** PDF 데이터 추출 및 증강 파이프라인
- Collaboration Tools:** GitHub (버전 관리), Notion (실험 설계 공유), Slack (실시간 소통)

2. 프로젝트 팀 구성 및 역할

이름	역할
강한	Focal Loss, NEFTune, CoT (In-context-learning), CoT (Reasoning Dataset based FineTuning)
김성호	streamlit을 통한 데이터 세부 분석, 리더보드를 통한 모델 선정 실험, unsloth 라이브러리를 통한 학습 환경 구축
김수영	PDF 추출 기반 데이터셋 구축, 프롬프트 순서 관련 실험
박찬서	harness를 활용 하여 한국어 벤치마크 성능이 가장 우수한 베이스 모델(7~10B)을 선별, 30B 모델(Qwen) 환경 구축
정지웅	LLM 기반 데이터셋 증강, llama.cpp 서버 환경 구축, MoA(Mixture-of-Agents) 유사 파이프라인 구현 및 실험

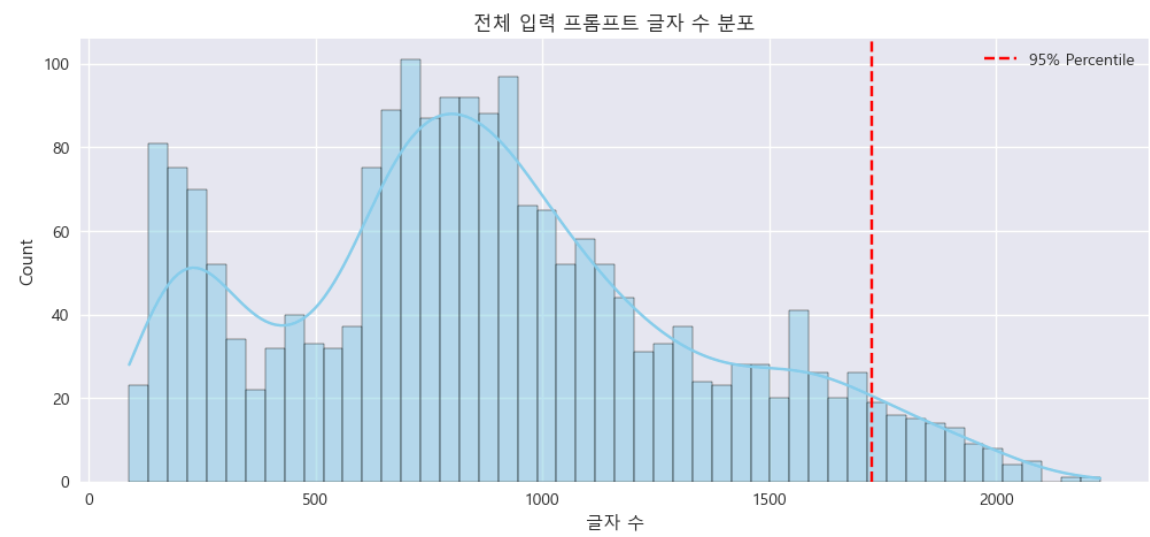
3. 프로젝트 수행 절차 및 방법론

3.1. 탐색적 데이터 분석 (EDA) 및 인사이트 도출

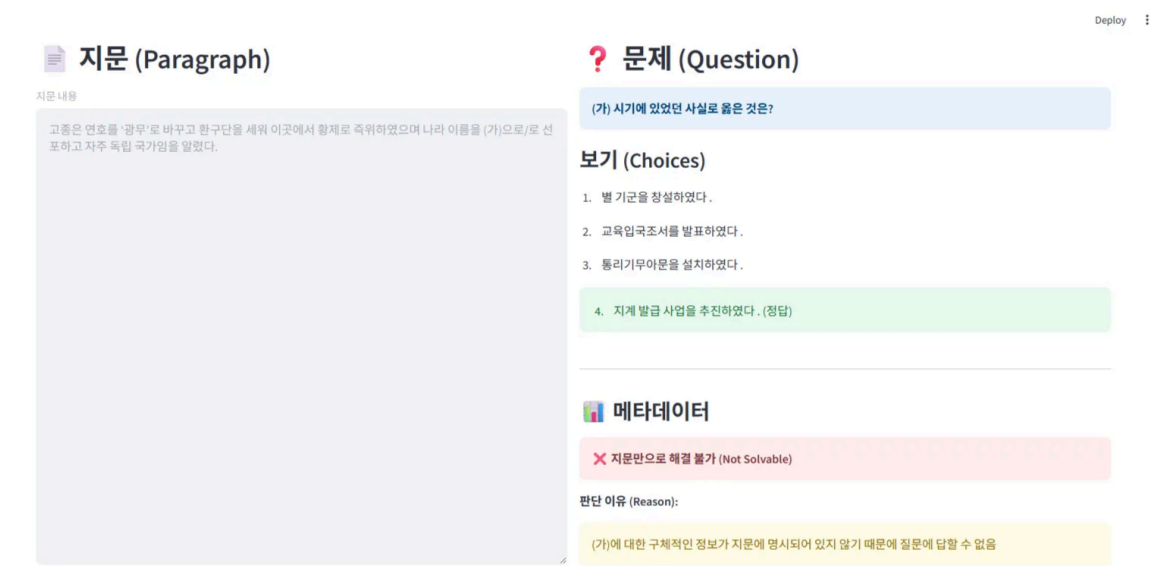
모델링 전략 수립에 앞서, 제공된 학습 데이터(Train: 2,031개, Test: 869개)의 특성을 정량적, 정성적으로 분석하였다. 이 과정에서 도출된 인사이트는 이후 실험 설계의 핵심 근거가 되었다.

3.1.1. 입력 길이 및 구조적 특성 분석

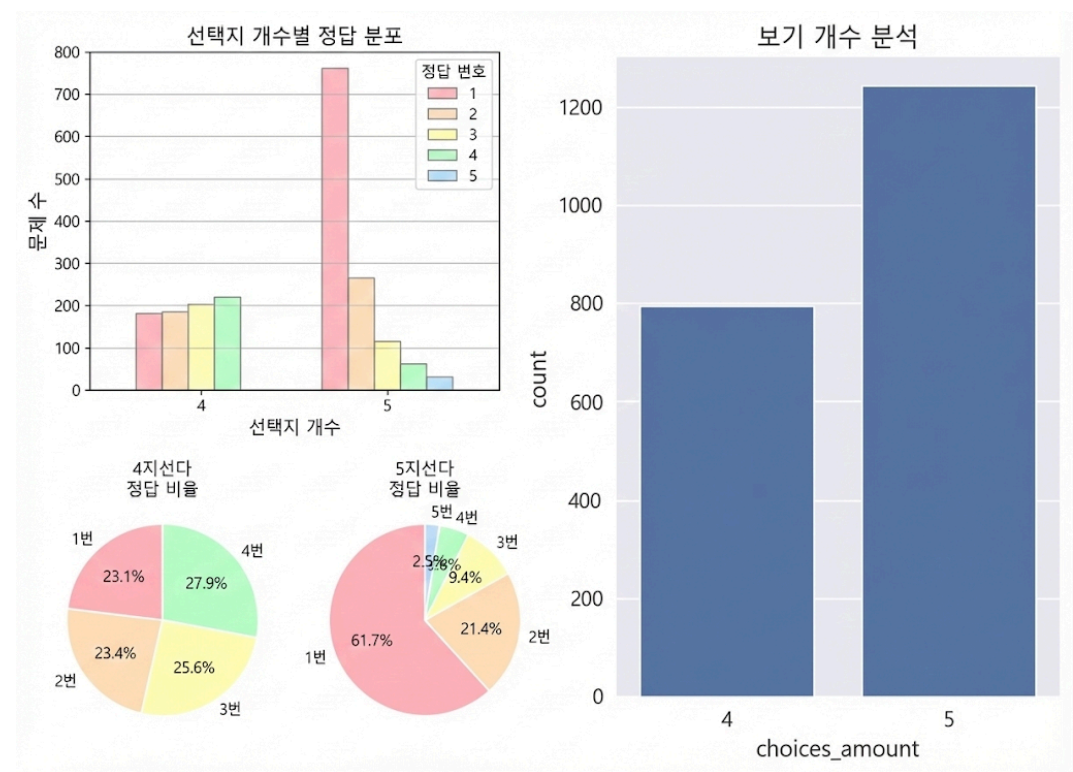
지문(Paragraph), 질문(Question), 선지(Choices)를 모두 합친 입력 텍스트의 길이를 분석한 결과, 대부분의 데이터가 일반적인 LLM의 Context Window 내에 포함됨을 확인하였다.



한편 gpt-4o-mini API와 Streamlit을 활용한 정성 분석 결과, 단순히 지문 내의 정보를 추출하는 것만으로는 풀 수 없는 '지식형 문제'가 상당수 존재함이 파악되었다. 이는 모델이 지문 외적인 외부 지식을 내재하고 있어야 함을 시사한다.



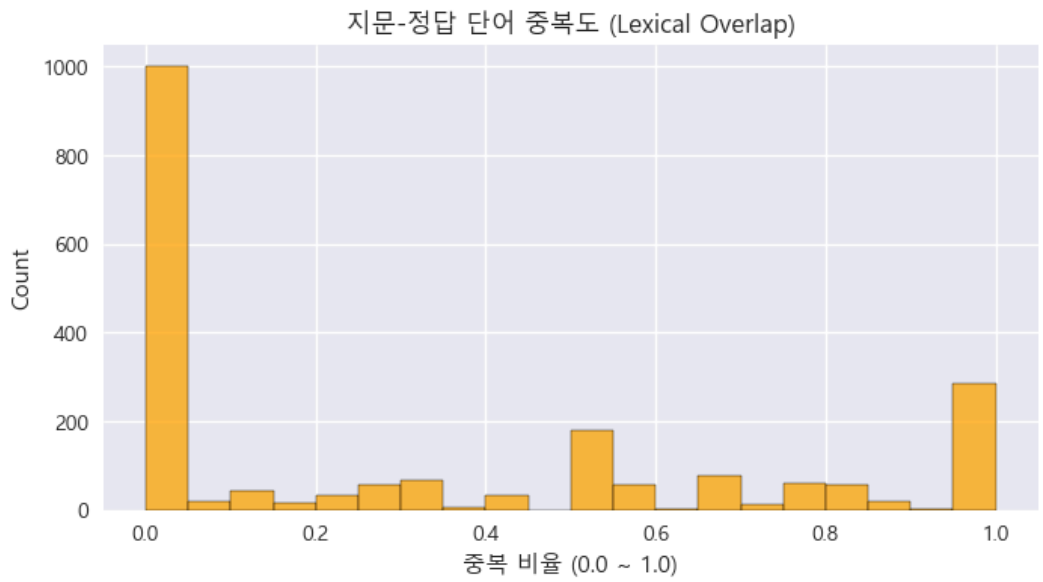
3.1.2. 선지 편향성의 발견



학습 데이터의 정답 분포를 분석한 결과, 극심한 클래스 불균형 현상이 발견되었다. 특히 5지선다형 문제의 경우, 1번이 정답인 비율이 전체의 약 61.7%를 차지하는 기형적인 분포를 보였다.

이러한 불균형은 모델이 문맥을 이해하고 정답을 추론하기보다, 단순히 1번을 선택하면 맞출 확률이 높다는 통계적 편향을 학습하게 만들 위험이 크다. 이는 테스트 데이터의 분포가 다를 경우 심각한 성능 저하로 이어질 수 있는 요인이다.

3.1.3. 지문-정답 단어 중복도(Lexical Overlap) 분석



'지문 내에 정답 키워드가 얼마나 등장하는가'를 나타내는 Lexical Overlap을 분석한 결과, 대부분의 데이터에서 중복도가 0.0에 가깝게 나타났다.

이는 수능형 문제가 단순한 패턴 매칭이나 기계적 독해로는 해결할 수 없음을 정량적으로 증명한다. 즉, 모델은 지문의 내용을 이해하고, 이를 바탕으로 논리적 추론을 거쳐 지문에 명시되지 않은 정답을 도출해야 한다는 것을 파악했다.

3.2. 데이터 증강 및 외부 데이터 활용 전략

3.2.1. 데이터 증강의 필요성

Llama-3.1, EXAONE-3.5 등 최신 SOTA Open LLM을 대상으로 **LM Evaluation Harness** 기반의 정량적 벤치마크 테스트를 수행하였다.

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
kmmlu_economics	2	none	0	acc	0.6923 ± 0.0406	
kmmlu_korean_history	2	none	0	acc	0.3900 ± 0.0490	
kmmlu_law	2	none	0	acc	0.5800 ± 0.0156	
kmmlu_political_science_and_sociology	2	none	0	acc	0.7133 ± 0.0262	
kmmlu_social_welfare	2	none	0	acc	0.6990 ± 0.0145	
kobest_boolq	1	none	0	acc	0.9302 ± 0.0068	
		none	0	f1	0.9301 ± N/A	
kobest_copa	1	none	0	acc	0.8210 ± 0.0121	
		none	0	f1	0.8209 ± N/A	
kobest_hellaswag	1	none	0	acc	0.5300 ± 0.0223	
		none	0	acc_norm	0.6060 ± 0.0219	
		none	0	f1	0.5258 ± N/A	

평가 결과, 일반 상식 영역 대비 **kmmlu_law** (법), **kmmlu_korean_history** (한국사) 등 한국 특화 도메인에 대해 약점을 보였다. 또한 **kobest_hellaswag** (긴 문맥 파악 및 상황 추론) 도메인에서도 약점을 보였다.

이를 통해, 부족한 **한국어 도메인 지식(Knowledge)** 주입 및 **문맥 추론 능력(Context Reasoning)** 강화를 위해 **도메인 특화 데이터 증강**을 결정하였다.

3.2.2. PDF 추출 기반 한국사 데이터셋 구축

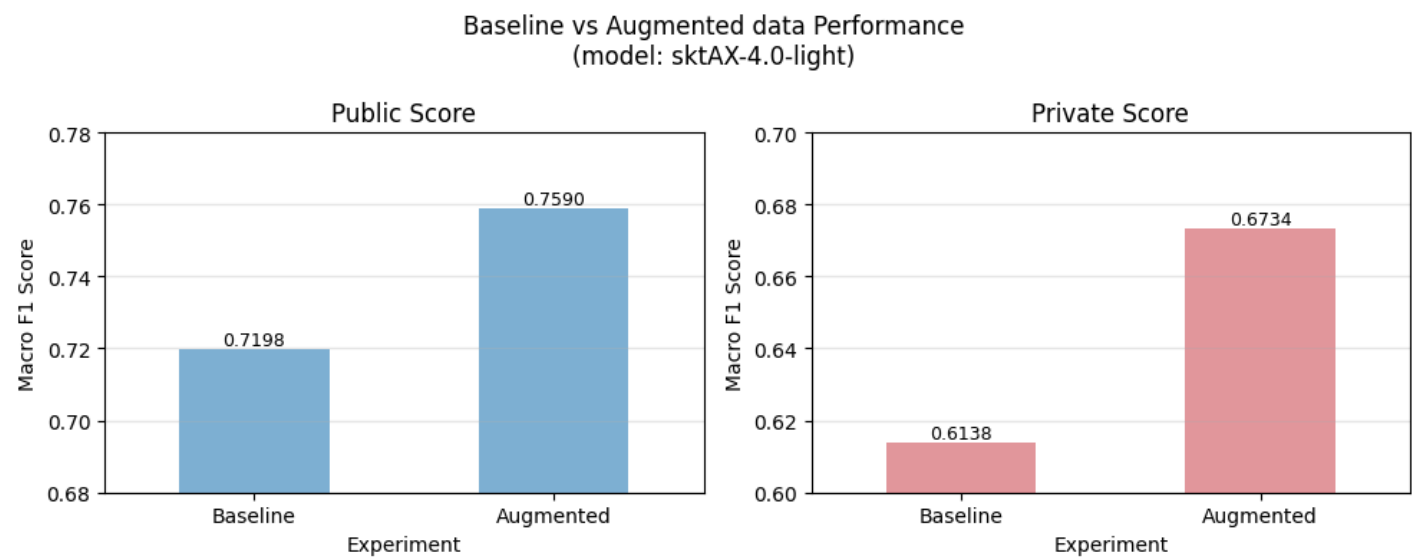
한국사 도메인의 데이터 부족을 해소하기 위해, 형식이 유사한 한국사능력검정시험(기본/심화) 기출문제를 학습 데이터로 변환하는 파이프라인을 구축하였다.

- 1. **Image Conversion:** **pypdfium2** 라이브러리를 사용하여 PDF 파일을 고해상도(300 DPI) 이미지로 변환, OCR 정확도를 확보하였다.
- 2. **Structured Extraction (GPT-4o Vision):** 멀티모달 모델인 GPT-4o Vision API를 활용하였다. 이를 통해 문제의 구조(지문, 질문, 선지)를 JSON 형태로 파싱하고, 특히 지도나 문화재 사진과 같은 시각 자료를 텍스트 묘사로 변환하여 언어 모델이 이해할 수 있는 형태로 가공하였다.
- 3. **Quality Assurance:** 추출 AI(GPT-4o)보다 더 성능이 좋은 AI로 문제를 풀게 하여, 틀리거나 풀 수 없다고 판단한 경우 해당 데이터를 제외하는 방식으로 품질 검수를 수행하였다.

3.2.3. LLM 기반 합성 데이터 생성

기존 데이터셋이 사실 관계 확인형 문제에 치중되어 있다는 점을 보완하기 위해, GPT-4o-mini를 활용하여 추론형 데이터를 생성하였다.

• **Method:** 뉴스 기사와 도서 말뭉치를 다양한 텍스트를 소스로 활용하고, <보기>를 포함한 복합 추론 문제, 부정형 질문("적절하지 않은 것은?") 등 기존 데이터에 부족한 유형을 생성하도록 프롬프트를 엔지니어링하였다. api 모델에 fine-tuning도 적용하여 생성하였으나 생성 데이터의 품질이 낮아 사용하지는 않았다.



• **Lesson Learned:** 생성된 데이터의 양적 팽창에는 성공했으나, 생성된 문항의 논리적 오류나 정답의 모호성을 걸러내는 품질 관리 (Quality Control) 프로세스가 미흡하였다. 이는 학습 데이터에 노이즈를 주입하는 결과를 초래할 수 있어, 추후에 품질 관리에 대한 필요함을 확인하였다.

3.3. 모델 선정 및 학습 환경 최적화

3.3.1. 베이스 모델 선정 (Model Selection)

한국어 처리 능력과 추론 능력을 종합적으로 평가하기 위해 LM Evaluation Harness 및 리더보드를 활용하여 다양한 오픈소스 모델을 벤치마킹하였다.

모델	F1 Score (public)
beomi/gemma-ko-2b	0.2672
beomi/Llama-3-Open-Ko-8B	0.5335
Qwen/Qwen2.5-7B-Instruct	0.6750
LGAI-EXAONE/EXAONE-3.5-7.8B-Instruct	0.6937
rtzr/ko-gemma-2-9b-it	0.6953
skt/A.X-4.0-Light	0.7173

그 결과 9B~11B 모델 중에서는 skt/A.X-4.0-Light가 가장 우수한 성능을 보였으나, 복잡한 추론 문제 해결에는 파라미터 사이즈의 한계가 존재함을 확인하였다.

3.3.2. 양자화(Quantization)를 통한 32B 모델 운용

제한된 V100 환경에서 더 큰 용량의 모델을 운용하기 위해 양자화 기술을 적극 도입하였다. Unsloth 라이브러리와 bitsandbytes의 4-bit Quantization 기술을 적용하여, Qwen2.5-32B-Instruct 모델을 V100 단일 카드에서 로드하고 학습시키는 데 성공하였다.

모델	F1 Score (public)
skt/A.X-4.0-Light	0.7173
unsloth/Qwen2.5-32B-Instruct-bnb-4bit	0.7779

이를 통해 모델의 추론 능력을 물리적 하드웨어 한계 이상으로 확장하였으며, F1 Score 역시 크게 향상한 것을 확인할 수 있었다. 학습 속도 또한 기존 HuggingFace Trainer 대비 2배 이상 향상시켜 제한된 시간 내에 더 많은 실험을 수행할 수 있는 기반을 마련하였다.

4. 실험 분석

본 프로젝트에서는 단순한 성능 경쟁을 넘어, LLM의 학습 및 추론 메커니즘을 이해하기 위한 다양한 실험을 수행하였다. 각 실험은 명확한 가설 하에 진행되었으며, 성공과 실패의 원인을 분석하여 최종 파이프라인에 반영하였다.

4.1. 실험 1: Focal Loss를 활용한 불균형 해소 시도

• **Hypothesis:** 데이터의 46.5%가 1번 정답인 상황에서, 모델이 쉬운 문제(1번)에만 과적합되는 것을 막고, 맞추기 어려운 소수 클래스 (High Entropy)에 가중치를 부여하면 성능이 향상될 것이다.

• **Method:** Gemma-2B 모델을 대상으로 **FocalLoss Trainer**를 커스텀 구현하였다. 정답 분포의 역수를 가중치(Alpha)로 설정하고, Gamma 값을 2.0으로 두어 확신하지 못하는 샘플에 대한 페널티를 강화하였다.

• **Result:** 성능 향상 없음 (Baseline과 유사하거나 소폭 하락).

Focal Loss는 이미지 분류와 같은 단일 출력 분류 태스크를 위해 고안된 손실 함수이다. 이미지 분류에서는 N개 클래스 중 1개를 선택하고, 그 단일 출력에 대해 Loss를 1회 계산한다. 반면 LLM은 "정답은 1번입니다"를 생성할 때 각 토큰("정답", "은", "1", "번" 등)마다 Loss가 계산되는 구조이다. 이러한 시퀀스 생성 방식에서는 단일 분류용 Focal Loss의 효과가 제한적이다. 또한 LLM의 vocabulary가 수만 개인 상황에서 정답 토큰("1","2","3","4","5")에만 가중치를 적용하는 것이 전체 학습에 미치는 영향이 미미하였다. 이는 LLM에서 클래스 불균형 해소는 Loss 기법보다 데이터 레벨(오버샘플링, 증강 등)에서 접근하는 것이 더 적합함을 시사한다.

4.2. 실험 2: NEFTune을 통한 과적합 방지

• **Hypothesis:** 적은 데이터셋으로 파인튜닝할 때 발생하는 과적합(Overfitting)을 막기 위해, 임베딩 레이어에 노이즈를 주입하면 일반화 성능이 개선될 것이다.

• **Method:** **NEFTune** 기법을 적용하여 학습 시 임베딩 벡터에 균등 분포(Uniform Distribution) 노이즈(alpha=5)를 추가하였다.

• **Result:** 성능 향상 없음.

NEFTune은 원논문에서 AlpacaEval 벤치마크 기준 +2~3점 향상을 보고하였으나, 이는 Instruction-following과 같이 다양한 표현이 정답이 될 수 있는 태스크에서 결과이다. 그러나 수능형 문제는 정확히 하나의 정답만 존재하므로 임베딩에 추가된 노이즈가 오히려 정밀한 추론을 방해하는 요소로 작용하였다.

또한, 모델이 특정 키워드를 보고 정답을 찍는 Shortcut Learning 문제를 해결하고자 하였으나 이는 임베딩 레벨이 아닌 어텐션 및 후속 레이어에서 발생하는 현상이므로 NEFTune만으로는 해결되지 않았다. 이는 도메인과 태스크의 성격에 따라 Regularization 기법을 선별적으로 적용해야함을 시사한다.

4.3. 실험 3: CoT (Chain-of-Thought) Fine-tuning

• **Hypothesis:** 모델이 단순히 정답을 맞추는 것이 아니라, "전제 파악 -> 선택지 검증 -> 결론 도출"이라는 논리적 추론 과정을 학습하면 정답률이 향상될 것이다.

• **Method:** GPT-4o-mini를 이용해 생성한 3,000개의 Reasoning 데이터셋(CoT 포함)으로 Qwen2.5-32B 모델을 QLoRA 파인튜닝하였다.

• **Result:** Baseline (F1 0.78) 대비 성능 급락 (F1 0.66).

ICL 방식의 CoT가 효과가 없었기에 파인튜닝으로 전환하려 하였으나 오히려 성능이 하락하였다. 이는 논문에서 "파인튜닝이 LLM의 CoT 추론 성능을 감소시킬 수 있다"라고 경고한 현상과 일치한다. 추가로 데이터 파이프라인 분석 결과, 원본 데이터 2,031개 중 75% (증강, Reasoning 데이터 선별 과정)가 소실되어 최종 학습에 508개만 포함되었고, 정답 분포가 1번 46.5%, 5번 1.7%로 심각하게 불균형하여 모델이 잘못된 패턴을 학습했을 가능성이 높다.

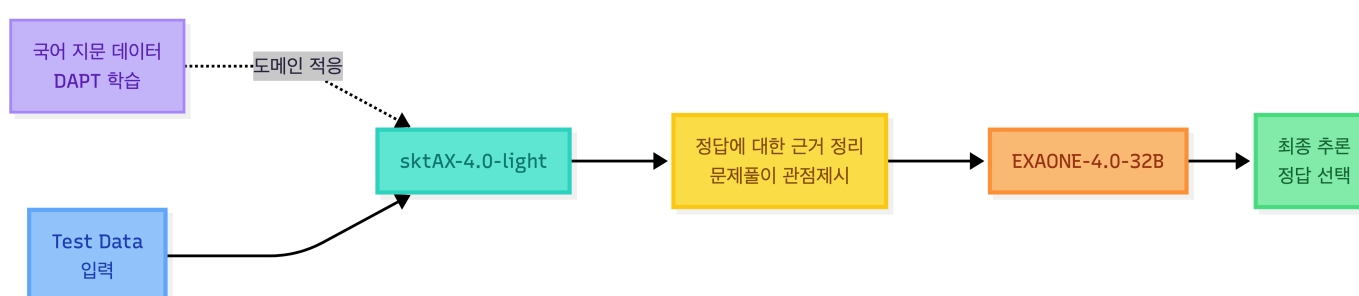
단순 SFT만으로는 CoT 능력을 효과적으로 학습시키기 어려우며, Qwen3 등 최신 모델들은 RL 기반의 복잡한 파이프라인을 사용한다. 최신 논문에 의하면, 100B 미만의 모델에서 모두 동일하게 CoT (ICL, Finetuning) 적용 시 성능이 하락하며, 새로운 정보를 학습할 때 이전에 학습한 정보를 급격하게 잊어버리는 현상인 catastrophic forgetting (파멸적 망각) 경향이 있었다.

결론적으로 ICL과 파인튜닝 모두 본 실험에서는 효과가 없었으며, 데이터 품질 확보와 1.7B ~ 75B 사이즈의 모델에서 좋은 효과를 보였던 DCoT 등 발전된 방법론 적용이 필요함을 느꼈다.

4.4. 실험 4: Mixture-of-Agents (MoA) 아키텍처 도입

• **Hypothesis:** 단일 모델의 추론 능력을 향상시키기 위해 추론 모델에 도움이 되는 정보를 추가제공하여 추론의 안정성을 높일 수 있다.

• **Method:** 제공된 GPU 환경에서 단일 pipeline을 구상할 때, 아래와 같은 MoA 파이프라인을 설계하였다.



◦ **Layer 1 (Proposers):** 경량화 모델(**skt/A.X-Light**)이 다양한 Temperature 설정으로 동일 문제에 대해 여러 관점의 힌트와 선택지 분석 정보를 생성한다.

◦ **Layer 2 (Aggregator):** 고성능 모델(**EXAONE-4.0-light**)이 Layer 1의 생성 정보도 포함하여 최종 정답을 결정한다.



- **Result: Partial Success.** 해당 아이디어를 늦게 적용하여 전체 데이터셋으로 진행한 결과는 얻지 못했다. 대신, 기존 결과들에서 예측이 엇갈리는 고난이도 문제셋(Subset)에만 MoA 적용하였고, 단일 모델 대비 Public/Private Score 모두에서 성능 향상과 안정성을 확인하였다.
- **Significance:** 모델 간의 역할 분담(Role-playing)을 통해 복잡한 문제를 해결할 수 있는 가능성을 보았다.

4.5. 계층적 앙상블 (Hierarchical Voting)

최종 결과물 제출을 위해, 개별 실험에서 얻은 모델들을 결합하는 앙상블 전략을 수립하였다. 단순 다수결(Hard Voting)의 경우, 성능이 낮은 모델들의 투표가 정답을 왜곡할 수 있다는 점을 보완하기 위해 '**신뢰도 기반 계층적 필터링**' 로직을 적용하였다.

- **Logic:** 전체 모델의 투표 결과가 과반수를 넘지 못하는 불확실한 경우, 성능이 검증된 리더 모델(Leader Models: Qwen2.5, Qwen3)의 합의를 최우선으로 따르며, 이들 간에도 의견이 갈릴 경우 Logits(Confidence Score)가 더 높은 쪽을 선택한다.

5. 프로젝트 수행 결과

순위	팀이름	팀멤버	MACRO_F1	제출횟수	최종 제출
내등수 9	NLP_16조		0.8073	36	1d

순위	팀이름	팀멤버	MACRO_F1	제출횟수	최종 제출
내등수 8	NLP_16조		0.7370	36	1d

Public 기준 9위, 최종 순위로는 8위를 차지하였다.

6. 자체 평가 의견

6.1. 좋았던 점

- 깃허브 협업 관련해서 토론을 하며 브랜치 전략을 그려가며 계획했던 게 좋았다.
- 프로젝트를 진행하며 나온 아이디어를 전부 정리한 뒤에, 이 정리한 문서를 바탕으로 실험과 프로젝트 내부 과정을 설계한 것이 좋았다.
- 각 실험을 진행한 것들에 대해 문서화를 중간중간 진행해놓아, 다른 팀원들이 진행한 실험의 현 상황에 대해 잘 알 수 있었고, 이를 추후 멘토링 시간에 사용하여 멘토님과의 좋은 의사소통 시간을 가질 수 있었다.

6.2. 아쉬웠던 점 및 개선할 점

- 관련 내용 논문이나 실험 결과들을 참고하여 실험해 보았으면 좋았을 거 같지만 잘 하지 못해서 아쉽다.
- 학습이나 추론이 오래걸리는 문제를 줄일 수 있는 설정을 초반에 미리 해놨다면 좋았을 것 같다. 이번 프로젝트로 경험했기에 다음 프로젝트에선 그러한 환경 설정을 미리 할 수 있을 것 같다.

개인회고 - 강 한

학습목표

파인튜닝을 통한 성능 향상 유도 및 실패 시에도 명확한 인사이트 도출에 집중

CoT 파인튜닝 실패 분석

주요 실패 원인

1. 원본 데이터 75% 소실 - train.csv 2,031개 중 508개만 학습에 사용. 증강 과정 407개(20%), 비율 완화 샘플링 1,116개(69%) 누락으로 학습-테스트 분포 불일치 심각
2. 정답 분포 불균형 - 1번 46.5%, 5번 1.7%로 극심한 불균형. 모델이 "잘 모르면 1번" 전략 학습
3. 단순 SFT 한계 - CoT 능력 학습에는 Long CoT Cold Start, Reasoning RL 등 복잡한 파이프라인 필요 (Qwen3 사례)

논문 근거 핵심

- 파인튜닝 위험성: 파인튜닝이 오히려 CoT 성능 감소시킴, 특히 작은 모델에서 Catastrophic Forgetting 유발 (On the Impact of Fine-Tuning on CoT Reasoning)
- 모델 크기 의존성: CoT 프롬프팅은 100B+ 모델에서 효과적, 32B는 불확실 영역 (Wei et al.)
- 개선 방향: DCoT가 1.3B~70B 전 범위에서 일관된 성능 향상 (Fine-Tuning with Divergent CoT)

교훈

데이터: 원본 데이터 100% 포함 필수, 정답 분포 균형, 증강 품질 검증
방법론: 단순 CoT SFT → DCoT 고려, 70B+ 모델 사용, 프롬프팅만 활용도 고려
프로세스: EDA 선행, 작은 모델 빠른 검증 후 스케일업, 체크포인트별 모니터링

Focal Loss 실패 분석

핵심 문제

태스크 불일치 - Focal Loss는 단일 출력 분류용. LLM은 토큰 단위 시퀀스 생성으로 "정답은 1번입니다" 각 토큰마다 Loss 계산되어 효과 제한적
가중치 영향 미미 - 수만 개 vocabulary 중 "1"~"5" 토큰에만 가중치 적용으로 전체 학습 영향 미미

교훈

분류 vs 생성, 단일 출력 vs 시퀀스, 기법의 원래 용도 확인 필수. LLM 클래스 불균형은 Loss보다 데이터 레벨(오버샘플링, 증강)에서 해결

NEFTune 실패 분석

핵심 문제

태스크 불일치 - NEFTune은 Instruction-following(다양한 표현 허용)에 효과적. 수능형 문제는 정확히 하나의 정답만 존재하여 노이즈가 정확한 추론 방해

Shortcut Learning 미해결 - 임베딩 레벨 노이즈로는 attention/추론 레벨에서 발생하는 Shortcut 패턴 해결 불가

교훈

태스크 특성, 정답 다양성, 논문 실험 환경 확인 필수. Shortcut Learning은 임베딩보다 데이터 레벨(패턴 다양화, 선택지 셔플링)에서 해결

종합 성찰

근본 문제: 한 가지 태스크에 몰두하여 CoT 집착, 검증된 논문/자료 통한 조기 방향 전환 실패

아쉬운 점: DCoT 논문을 더 빨리 발견했다면 시도 가능했으나, 탐색적 실험에만 리소스 소진

다음 프로젝트 원칙

1. 한 가지에 과도한 몰두 지양, 리소스 분산 투자
2. 사전 검증 철저화 (논문 근거, 태스크 적합성)
3. 탐색적 실험 최소화, 검증된 방법론 우선 적용

개인회고 - 김성호

학습 목표

이번 프로젝트는 수능형 문제를 해결할 수 있는 모델을 만들어보자는 Task에 대해 수행했다.

지난 프로젝트 이후 팀 내에서 피드백을 진행하였다. 이후 이번 프로젝트에서는 데이터에 대해 조금 더 깊게 분석해보고 그에 맞는 전략을 세워 보기로 하였다. 특히 기본적인 모델의 성능이 일정 수준 이상이어야, 우리가 원하는 것이 잘 작동할 것으로 생각하였다. 그를 위해 사용할 수 있는 모델을 많이 찾아보며 기본적인 성능을 높이는 것도 중요하다고 생각했다.

나는 무엇을 수행했는가?

먼저 데이터에 대한 분석을 진행했다. 그 과정에서 특히 문제의 유형을 중점으로 분석하였다.

이후 학습 과정에 사용할 모델을 테스트하였으며, 더 큰 모델의 용이한 사용을 위해 unsloth 라이브러리를 사용한 코드를 학습에 적용하였다.

같은 팀원들이 시행한 실험에 대해 추가로 분석하며, 어떠한 점에서 개선 방향성이 있을지 고민하기도 하였다.

나는 어떻게 수행했는가?

팀 회의를 진행하며 데이터를 직접 살펴보면 지문이 어떠한 형태로 되어있는지 알 수 있고, 문제의 카테고리를 어느정도 이해하는 것이 이후 설계에 도움이 될 것으로 판단하였다. 이를 직접 살펴보기 위해 다양한 방법을 사용했다. Streamlit을 이용해 데이터를 관찰하면서 이 문제가 독해형 문제인지, 지식형 문제인지 파악하는 과정을 거쳤다.

데이터에 대한 분석이 어느정도 완료된 뒤에는, 모델에 대한 실험을 진행하였다. 모델 선정에는 네이버에서 제공한 리포트를 참고하여 모델을 선정할 수 있게 되었고, 제공받은 서버에서 작동 가능한 적당한 규모의 모델 내에서 선택을 하게 되었다. 이후 양자화한 모델의 존재를 추가로 알게되어, 최종적으로는 unsloth 사에서 제공한 Qwen2.5-32B 모델을 4bit로 양자화한 모델을 최종적으로 선정하게 되었다. 모델을 선정하는 데에 시행한 실험은 제공된 베이스라인에 Chat Template만 해당 모델에 맞추어주는 방식으로 진행하였다. 한가지 알게된 것은, 모델의 기본적인 파라미터 수를 늘리는 것이 작은 모델에서 여러 방법론을 적용해보는 것보다 훨씬 높은 성능 향상이 있었음이 확인되었다.

멘토링 이후, 큰 모델을 학습하지 않고 In-Context Learning을 진행해보라는 이야기를 들어 추가로 해당 모델에 대한 실험을 진행해보았다. 다만 이때에는 기존처럼 logit에서 확률이 가장 높은 토큰을 출력하는 것이 아닌, 실제로 토큰을 생성하는 방식으로 코드를 수정하여 실험을 진행하였다.

아쉬웠던 점은?

가장 아쉬웠던 것은 많은 실험을 진행해보지 못 한 것이다. 모델의 사이즈를 32B로 늘리면서 학습 및 추론에 소모되는 시간이 크게 늘어났는데, 현존하는 라이브러리 중에서는 이러한 모델을 사용하는 데에 드는 시간을 줄이는 다양한 라이브러리가 존재한다. 그러나 이를 적용하는 것이 기존의 작업하던 파일들과 호환성이 맞지 않을 수 있어 우선순위를 뒤로 미룬 것이 가장 아쉬웠다. 그러한 환경을 먼저 설정하고 난 뒤, 다양한 실험을 해봤다면 모델에 대해 여러가지 실험을 해볼 수 있었을 것이고, 그렇다면 더 좋은 결과가 나오지 않았을까 싶다.

멘토링 때에도 이야기를 들었지만, 내가 세운 가설의 올바름을 증명하기 위해서는 실험의 결과를 보아야 알 수 있다. 특히나 이번 프로젝트처럼 가용한 자원이 한정적일 때에는, 그러한 실험 설계를 더욱 정밀하게 진행하고, 작은 규모의 모델에서 실험을 진행해보는 것이 더욱 효과적일 것이라는 판단을 내렸다.

전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고 어떤 효과가 있었는가?

우선 프로젝트 관리를 위한 도구를 많이 사용했던 점이였다. Hydra를 통하여 실험에 진행할 Config를 잘 관리하였고, 노션 및 슬랙을 통해 협업 과정에서 팀원들과 공유할 것들을 적극적으로 이야기하고 문서화했다는 점이 가장 큰 변화였다.

전 프로젝트에서는 단순히 어떠한 기능을 구현하고, 이런 것들을 해보면 성능향상으로 이어지지 않을까? 라는 목표에 사로잡혀, 단순히 해당 프로젝트를 여타 다른 프로젝트와 마찬가지로 대했던 것 같다. 이번 프로젝트에서는 실험에 대해 많은 고민을 진행하였고, 그 가설을 검증해 내기 위한 설계를 체계적으로 진행해보려 했던 것 같다.

이번 프로젝트에서 느낀 점은?

사실 이번 프로젝트는 단순히 어떠한 Task를 수행하기 위해 이러한 파이프라인을 설계하고, 이러한 것들을 진행해야 한다! 라는 것이 주목적이지 아니었다라는 생각을 하기도 했다. ChatGPT나 Gemini와 같은 웹으로 사용할 수 있는 생성형 AI 서비스에서는, 이러한 문제를 푸는 것이 이미 충분히 검증된 Task이기도 했기 때문이다. 결국 생각했던 문제는 **"작은 모델은 큰 모델을 이길 수 없는가?"** 가 생각으로 맴돌기도 하였다.

한편 특히나 우리에게 주어진 자원이 한정적이었다는 점이, 앞으로 엔지니어로써 겪을 문제 중 하나라고 생각을 하였다. 주어진 자원이 한정적이라면, 그 자원을 최대한으로 활용하며 큰 모델을 사용하는 것이 이 프로젝트에서 전하는 메시지라고 생각했다.

개인회고 - 김수영

1. 나는 내 학습목표를 달성하기 위해 무엇을 어떻게 했는가?

이번 프로젝트에서 나는 크게 두 가지 과제를 담당하였다. 첫째는 한국사 도메인 데이터 부족 문제를 해결하기 위한 **한국사능력검정시험 데이터셋** 구축이었고, 둘째는 LLM의 긴 컨텍스트 처리 특성을 활용한 **프롬프트 순서 최적화 실험**이었다.

1. 한능검 데이터셋 구축

기존에 제공된 train 데이터에서 한국사 영역의 샘플이 부족하다는 문제를 인식하고, 외부 공인시험 데이터를 활용한 데이터 증강을 시도하였다. 단순 크롤링이 아닌 GPT-4o Vision을 활용하여 OCR과 구조화 추출을 동시에 수행함으로써, 이미지가 포함된 한국사 문제도 텍스트 기반 학습 데이터로 변환할 수 있었다. 이는 수작업 대비 시간과 노력을 크게 절감하는 성과였다. 다만, 구축한 데이터셋이 최종 모델 성능 향상에 얼마나 기여했는지는 명확히 검증하지 못한 한계가 있다.

2. 프롬프트 순서 변경

Lost in the Middle 논문의 핵심 발견인 "LLM은 긴 컨텍스트에서 앞과 뒤 정보를 잘 기억하고, 중간 정보는 놓치기 쉽다"는 U-shaped attention 패턴을 수능형 문제 풀이에 적용하고자 하였다. 질문과 선택지라는 핵심 정보를 앞과 뒤에 배치하고, 상대적으로 긴 지문을 중간에 배치하는 전략을 적용하였다. 기존 베이스라인(Qwen2.5-32B, F1: 0.0999) 대비 오히려 성능이 소폭 하락하였다(Qwen3-32B, F1: 0.0873). 논문에서 제시한 이론과 실제 실험 결과가 다르게 나타난 것이다.

2. 마주한 한계와 아쉬웠던 점

가장 크게 와닿은 것은 멘토님께서 해주신 "빠른 실험, 선택과 집중"이라는 말씀이다. 정해진 리소스(GPU, 시간) 안에서 좋은 결과를 내려면, 빠르고 간단한 실험을 통해 해당 접근법의 가능성을 먼저 파악하는 것이 중요하다. 가능성이 확인되면 그때 프로젝트의 방향성을 잡고 본격적으로 리소스를 투입하여 결과를 내는 것이다. 이 부분이 내가 가장 잘 못한 부분이었다. 한번의 실험을 검증하는 데 32B 모델로 3 epoch을 돌리니 무려 36시간이 걸렸다. 하나의 실험에 이를 가까이를 소비하니 다양한 실험을 시도해볼 여유가 없었다.

결과적으로 "논문대로 큰 모델로 제대로 해보자"는 접근이 오히려 실험의 다양성을 제한하고, 명확한 결론을 내리지 못하게 만든 원인이 되었다. 프롬프트 순서만 변경한 ablation study, 데이터셋만 변경한 실험, 모델만 변경한 실험 등을 각각 수행했어야 했는데, 긴 학습 시간 때문에 이러한 체계적인 비교가 불가능했다.

또한 한능검 데이터셋을 구축하는 데 상당한 시간을 투자하였으나, 이 데이터가 실제로 모델 성능 향상에 기여했는지 검증하는 실험을 수행하지 못한 점도 아쉽다. 데이터 구축과 검증이 별개의 작업이 아니라 하나의 사이클로 이루어져야 한다는 교훈을 얻었다.

3. 다음 프로젝트에서 시도해보고 싶은 점

1. 빠른 실험, 선택과 집중

다음 프로젝트에서는 "작은 모델로 빠르게 검증하고, 가능성이 보이면 스케일업한다"는 원칙을 반드시 지키고 싶다. 새로운 아이디어가 떠오르면 바로 32B 모델에 적용하는 것이 아니라, 먼저 7B~8B 규모의 모델로 실험하거나 혹은 1 epoch만 돌려서 경향성을 파악하는 것이다. 이 단계에서 가능성이 보이면 리소스를 집중 투입하고, 효과가 없어 보이면 빠르게 다른 접근법으로 전환하는 유연함을 갖추고 싶다.

2. 데이터 분석 우선 접근

데이터 증강이나 모델 변경에 앞서, 기존 데이터에 대한 심층적인 분석을 먼저 수행하는 접근법을 시도하고 싶다. 이번 프로젝트에서는 클러스터링 기반 EDA를 수행하였으나, 그 결과를 바탕으로 한 전략 수립이 충분하지 않았다. 다음에는 "어떤 유형의 문제에서 모델이 약한가?", "오답 패턴은 무엇인가?" 등 여러 분석을 철저히 수행한 뒤, 그에 맞는 타겟팅 개선 전략을 수립하고 싶다.

3. 시도하지 못한 방법들

시간 관계상 시도하지 못한 방법들도 향후 프로젝트에서 적용해보고 싶다. 첫째, OCR 추출 시 RAG를 활용하여 도메인 특화 사전(한국사 인물 백과, 용어 사전)을 참조하도록 하는 방식이다. 둘째, OCR confidence 기반의 품질 관리 로직을 추가하여 낮은 신뢰도의 데이터를 자동으로 필터링하는 방식이다. 이러한 방법들이 데이터 품질 향상에 얼마나 기여하는지 검증해보고 싶다.

4. 프로젝트를 마치며

이번 프로젝트는 "이론과 실재는 다르다"는 교훈을 몸소 체험한 경험이었다. Lost in the Middle 논문의 발견이 명확하고 논리적으로 보였기에 프롬프트 순서 변경이 당연히 성능 향상으로 이어질 것이라 기대하였으나, 실제 결과는 그렇지 않았다. 논문의 실험 환경과 내 실험 환경의 차이(모델 종류, 데이터 특성, 태스크 유형 등)가 결과에 영향을 미쳤을 수 있으며, 이러한 차이를 충분히 고려하지 못한 점이 아쉽다.

무엇보다 이번 프로젝트를 통해 빠른 실험과 선택과 집중의 중요성을 뼈저리게 느꼈다. 제한된 리소스 안에서 최대의 성과를 내려면, 모든 것을 완벽하게 하려고 하기보다는 빠르게 가능성을 검증하고 방향을 잡는 것이 훨씬 효과적이라는걸 깨달았다.

결과적으로 이번 프로젝트에서 눈에 띄는 성능 향상을 달성하지는 못하였으나, 실패를 통해 배운 점이 많았다. 체계적인 실험 설계의 중요성, 변수 통제의 필요성, 빠른 검증과 선택과 집중의 가치, 데이터 품질 관리의 어려움 등은 앞으로의 프로젝트에서 더 나은 접근을 할 수 있는 밑거름이 될 것이라 믿는다. 무엇보다 "일단 시도해보는 것"의 가치를 깨달았고, 비록 결과가 기대와 달랐더라도 그 과정에서 얻은 경험과 인사이트는 소중한 자산이 되었다.

개인회고 - 박찬서

1. 학습 목표

제한된 GPU(V100, 32GB)와 스토리지(50GB) 환경에서 어떻게 30B 규모의 모델을 구동하고, 작은 모델로도 최대한의 성능을 이끌어 내는 것 이었습니다.

- 정량적 기준에 따른 베이스 모델 선정: lm-evaluation-harness를 도입해 한국어 벤치마크 성능이 가장 우수한 베이스 모델을 선별했습니다.
- 자원 효율화: Hugging Face의 Files 탭을 통해 모델 파라미터 용량을 사전에 계산하고 BitsAndBytes의 4bit Quantization(QLoRA) 과 Gradient Checkpointing 기술을 적용해 OOM문제를 해결했습니다.

2. 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고 어떤 효과가 있었는가?

- Hydra를 통한 실험 자동화: 하드코딩된 설정값으로 매번 코드를 수정하던 비효율을 개선하고자 Hydra를 도입했습니다. Config 파일을 모듈화하여 다양한 하이퍼파라미터 조합을 동적으로 실험할 수 있는 파이프라인을 구축했습니다.
- 한국어 토큰나이징: 초기에는 전통적인 NLP 방식대로 형태소 분석기를 통해 조사를 분리하려 했습니다. LLM 학습에서는 조사를 포함한 문장 전체의 문맥을 학습하는 것이 자연스러운 생성과 추론에 훨씬 유리함을 깨달았습니다. 이를 통해 과도한 전처리로 인한 정보 손실을 막고 모델의 언어 이해도를 높였습니다.

3. 마주한 한계와 아쉬운 점

- 데이터 선별의 정밀성 부족: Harness 평가를 통해 모델이 한국사와 법률 도메인에 취약함을 발견했습니다. AI Hub에서 관련 데이터를 확보했으나, 단순히 양적인 증강에 집중하느라 kmmlu_law의 세부 과목(민법, 형법 등)별 약점을 파악하여 타겟팅된 데이터를 구축하지 못한 점이 아쉬웠습니다.
- 과적합과 리소스 관리의 딜레마: 추론 시간이 오래 걸리는 30B 모델 특성상 충분한 실험을 하지 못해, 3 Epoch 학습 시 Overfitting으로 리더보드 점수가 하락하는 경험을 했습니다. 결과적으로 1 Epoch 모델이 성능이 가장 좋았는데, 이는 데이터 양보다 질, 그리고 적절한 학습 시점 종료가 중요함을 느꼈습니다.
- Reasoning 구현 실패: Few-shot(20개)을 통한 추론을 시도했으나, max_length 초과로 인한 OOM 문제를 해결하지 못했습니다.
- 파인튜닝한 모델을 CoT를 돌리니 성능이 오히려 떨어졌습니다.

4. 교훈을 바탕으로 한 향후 계획

- Logit 기반의 정밀 진단: 다음에는 Harness의 Logit 값을 분석하여 특정 도메인의 세부 카테고리별 정답률을 확인하고, 정확히 그 부분의 데이터만 증강하는 방법론을 적용하겠습니다.
- 메모리 최적화 심화: OOM으로 포기했던 Few-shot Reasoning을 구현하기 위해, KV Cache 관리 등 더 고도화된 메모리 최적화 기법을 연구하여 긴 문맥을 다루는 능력을 확보하고 싶습니다.
- 파인튜닝에만 몰두하지 말고 Reasoning 시도를 많이 해보고 논문 연구도 참고하여 실험을 진행해 보겠습니다.

개인회고 - 정지웅

1. 협업의 시도와 생각

이전 프로젝트에서 협업과 팀플레이가 상당히 아쉬웠던 경험이 있어, 이번 프로젝트를 시작할 때는 **협업 구조를 제대로 갖추는 것에** 많은 신경을 썼다. 공통 대시보드를 만들고, 실험 결과를 공유할 수 있는 공간을 마련하는 등 시작 단계에서는 비교적 체계적인 협업 환경을 구축했다고 생각한다.

다만 프로젝트를 진행하면서, 이번 과제의 특성상 **실험과 모델 실행에 시간이 많이 소요되고 빠른 반복이 요구되는 구조는 아니었기 때문에**, 협업 시스템을 잘 구축한 것 자체가 곧바로 큰 효율로 이어지지 않는다는 생각이 들었다. 돌이켜보면 이러한 협업 시스템은 오히려 지난 프로젝트에서 더 절실히 필요했던 요소였다는 생각도 든다.

그럼에도 불구하고, 협업을 '잘하려는 시도'를 의식적으로 했다는 점이 좋았고, 팀플을 하는 과정도 나름 안정적이지 않았나 생각한다.

2. 잘했던 점

이번 프로젝트에서 가장 마음에 들었던 점은, 이전 프로젝트와 달리 과거 기수의 결과나 리포트를 참고하지 않고 프로젝트를 진행했다는 점이다. 물론 선행 사례를 참고하는 과정도 필요하고 중요하지만, 지난 번 프로젝트에서 이전 기수들의 방식에 매몰되었던 전적이 있어서 이번에는 그러한 영향을 의도적으로 차단하였다.

대신, 문제를 어떻게 바라볼 것인지에 대한 기준을 스스로 세우는 데 집중했다. 남의 풀이를 따라가기보다는 내가 무엇을 목표로 하고 있는지, 문제 해결에 중요 포인트가 무엇이 될 지와 같은 판단 기준을 명확하게 하였다. 그 기준을 바탕으로 문제에 접근하고 아이디어를 구체화하면서 실험을 설계·진행할 수 있었다.

나에게는 이번 프로젝트는 단순히 점수나 순위로 평가되는 프로젝트가 아니라, 문제를 바라보는 관점과 실험을 설계하는 기준을 스스로 세워보고 검증해본 경험으로 남았다는 점에서 개인적으로 의미있었다. 많은 고민과 시행착오를 거쳐 만들어낸 결과인 만큼, 성과의 크기와 무관하게 이번 프로젝트에는 이전보다 더 큰 애정과 만족이 남아 있다.

3. 한계와 아쉬움

이번 프로젝트를 통해 스스로의 한계도 일부 확인할 수 있었다. 개방적으로 사고하려 노력했다고 생각했지만, 실제로는 특정 아이디어나 접근 방식에 의외로 강하게 고착되는 순간들이 있었다.

예를 들어, 데이터 QC 과정에서 데이터를 cherry picking 하면 안된다는 생각에 객관적으로 판단하지 못하고 저품질의 데이터를 계속 앓고 갔다는 것. 또 MoA 구조를 설계할 때도, "하나의 inference flow에서 결과를 뽑아내야 한다"는 전제를 자연스럽게 두고 소형 모델과 대형 모델을 1:1로 구성하는 방식에 집중했다. 물론, 접근은 **주어진 리소스 내에서 최적이고 시간적 비용도 낮다는 점에서 합리적이었지만**, 컴페티션이라는 맥락에서는 모델을 여러 번 올리고 내리거나, 여러 모델들의 결과를 누적·비교하는 전략도 충분히 고려해볼 수 있었다는 점에서 아쉬움이 남는다. 결과적으로 "한 스텝에서의 최적"에 지나치게 몰두했던 부분이 있었다고 느낀다.

참고문헌

- AI, 수능에 도전하다: KoNET
 - <https://clova.ai/tech-blog/ai-수능에-도전하다-konet>
- Mixture-of-agents enhances large language model capabilities (Wang, L., et al., 2024)
 - <https://arxiv.org/abs/2406.04692>
- Chain-of-Thought Prompting Elicits Reasoning in Large Language Models (2022)
 - <https://arxiv.org/abs/2201.11903>
- On the Impact of Fine-Tuning on Chain-of-Thought Reasoning (2024)
 - <https://arxiv.org/abs/2411.15382>
- Fine-Tuning with Divergent Chains of Thought Boosts Reasoning Through Self-Correction in Language Models (2024)
 - <https://arxiv.org/abs/2407.03181>
- Focal Loss for Dense Object Detection (2017)
 - <https://arxiv.org/abs/1708.02002>
- NEFTune: Noisy Embeddings Improve Instruction Finetuning (2023)
 - <https://arxiv.org/abs/2310.05914>
- Lost in the Middle: How Language Models Use Long Contexts (Liu et al., 2023)
 - <https://arxiv.org/abs/2307.03172>